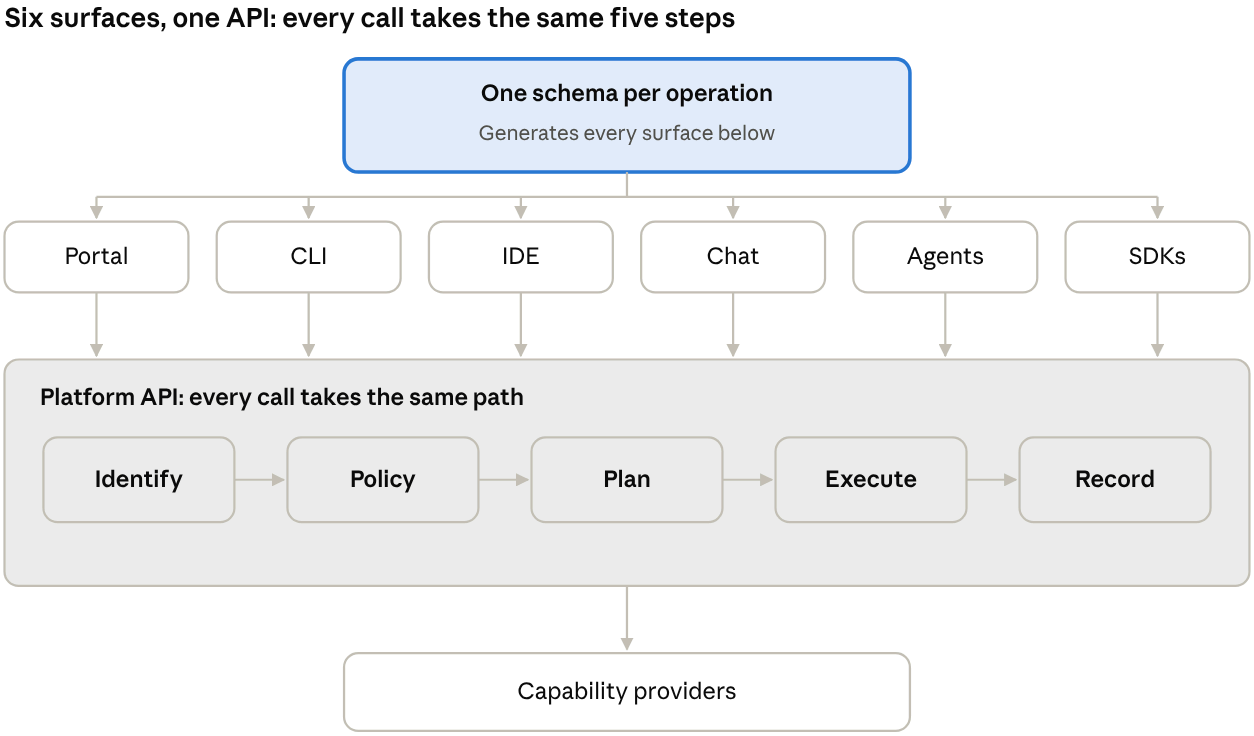
One Backend, Many Surfaces

Oct 29, 2026 · @Stephen

Put all the logic in one API, describe every operation once in a schema, and generate each surface from that schema. A surface should only handle presentation: forms in the portal, flags in the CLI, tool definitions for agents. Once any surface contains its own logic, the surfaces start to drift apart.



one schema, six surfaces, one five-step API path

Every surface is generated from the same schema and calls the same API, so a rule added once applies everywhere.

Who the platform serves

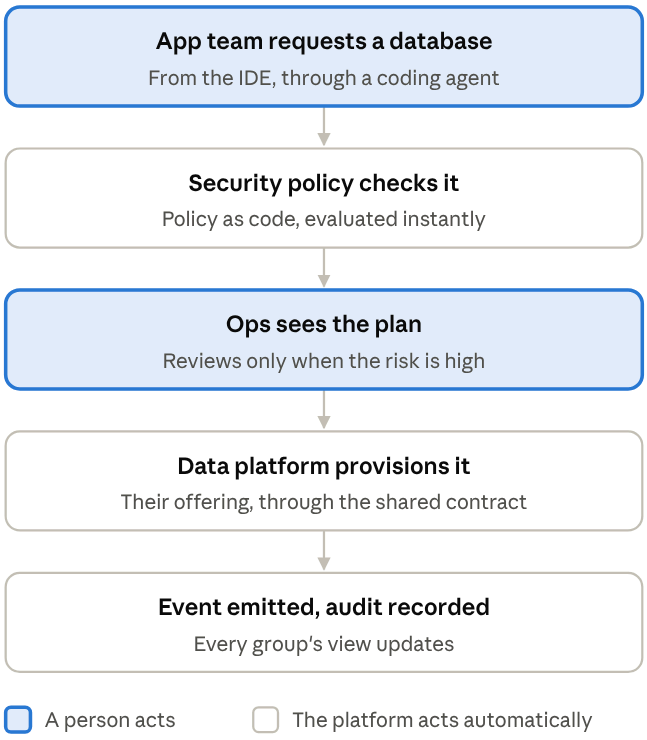
A portal that only helps app teams create a service on day one is a launcher, not a platform. Every group below should be able to do its routine work through the shared core, from the surface it already uses.

Group	Daily work	Where they work	What the shared core gives them
App teams	Ship, fix, request resources, check costs	IDE, CLI, pull requests, coding agents	Self-service through approved paths, a view of everything they own
Ops and SRE	Incidents, change review, capacity, routine fixes	Chat, on-call tools, dashboards, ops agents	A change calendar, blast radius from the entity graph, runbooks that actually run
Security	Policy, posture, exceptions, audit evidence	Policy repos, ticketing, compliance tools	Rules enforced at request time, exceptions that expire, a complete record of who did what
Platform teams	Build and run their offering (data, networking, ML)	Their own repos and services	A contract to publish into, so they get the same surfaces, policy, and events without rebuilding them
Agents	Act for any of the above	MCP and the API	Their own identity and limited permissions, with every action linked to the person who started it

Platform teams sit on both sides: they use the core like everyone else, and they plug their own offerings into it. That is what makes it a platform of platforms.

One change, every group

One change touches every group, and no one opens a ticket



one database request · five groups, one shared record

Each group does its part where it already works, and all of them see the same record. If only app teams use the platform, and only on day one, you've built a launcher. Track the share of changes that pass through the shared record, the routine ops tasks that run automatically, and the security exceptions that expire on time.

The schema is the product

Define every operation once: its inputs, outputs, errors, whether it can be undone, how far its effects reach, and the permissions it needs. Use OpenAPI 3.1 or JSON Schema, or Kubernetes-style resource definitions if you're declarative throughout. From that one definition, generate:

- **CLI:** commands, flags, help text, and shell completion
- **Portal:** forms, validation, and the action buttons on each entity page
- **Agents:** MCP tool definitions, with the safety annotations copied straight from the schema

- **SDKs:** typed clients for each language
- **Docs:** API reference pages, plus llms.txt entries

Adding a field then means editing one file, and every surface picks it up. That's what keeps them consistent.

Prefer declarative resources over a long list of actions

Model infrastructure as resources with a desired state ("this database should exist with these settings") rather than hundreds of action endpoints. The Kubernetes API is the best example. Declarative resources make plan and diff natural, and let the platform keep correcting drift.

They are also much safer for agents. An agent that proposes a desired state is easier to review than one firing a sequence of commands.

Every call takes the same path

These five steps live in the API, never in a surface:

1. **Identify:** know both the person and the surface they came through. Agents get their own identity plus a token delegated by the user who started them, so every action shows "agent X for person Y". OAuth token exchange is the standard way to do this.
2. **Policy:** evaluate rules on every request, from every surface, the same way. If the portal checks something the CLI doesn't, people will use the CLI to get around it.
3. **Plan:** every change supports dry-run and returns a diff. The portal shows it as a review screen, the CLI as `--plan`, and an agent shows it to the user before applying.
4. **Execute:** long-running work returns an operation ID right away, with status you can check or follow. Requests are safe to retry, so a timed-out agent can try again without creating duplicates.
5. **Record:** every operation writes to the audit log and emits an event. This keeps chat notifications, release updates, and every team's views in sync.

What each surface should and shouldn't do

Surface	Should	Should not
Portal	Show entity pages, run forms generated from the schema, offer a "copy as CLI command" button	Call other tools' backends directly, or contain logic of its own
CLI	Be generated, scriptable, and support plan, JSON output, and waiting for completion	Wrap one-off scripts that bypass the API
IDE	Offer validation and inline plans for config files	Hold separate credentials or a separate code path
Chat	Show approvals, notifications, and quick read-only lookups	Run changes without the same plan and approval step
Agents	Use the generated MCP tools with their safety annotations	Use a hand-written tool set that drifts from the API

The trap to avoid

A plugin-based portal makes it tempting for each plugin to call Jenkins, the cloud provider, or PagerDuty directly. That's how a portal ends up as a launcher: its behavior exists only in the portal. Plugins are fine for display, but any change should go through your platform API.

How to enforce it

- In CI, check that every operation has a CLI command, an MCP tool, and docs. Fail the build if any is missing.
- Give surfaces no direct credentials to downstream systems. Only the API's executor holds them, which makes going around the API impossible, not just discouraged.
- Use the same operation names everywhere. If the portal says "Rotate credentials", the CLI command is `rotate-credentials` and the MCP tool `rotate_credentials`. Agents and people both learn the platform faster when the vocabulary matches.